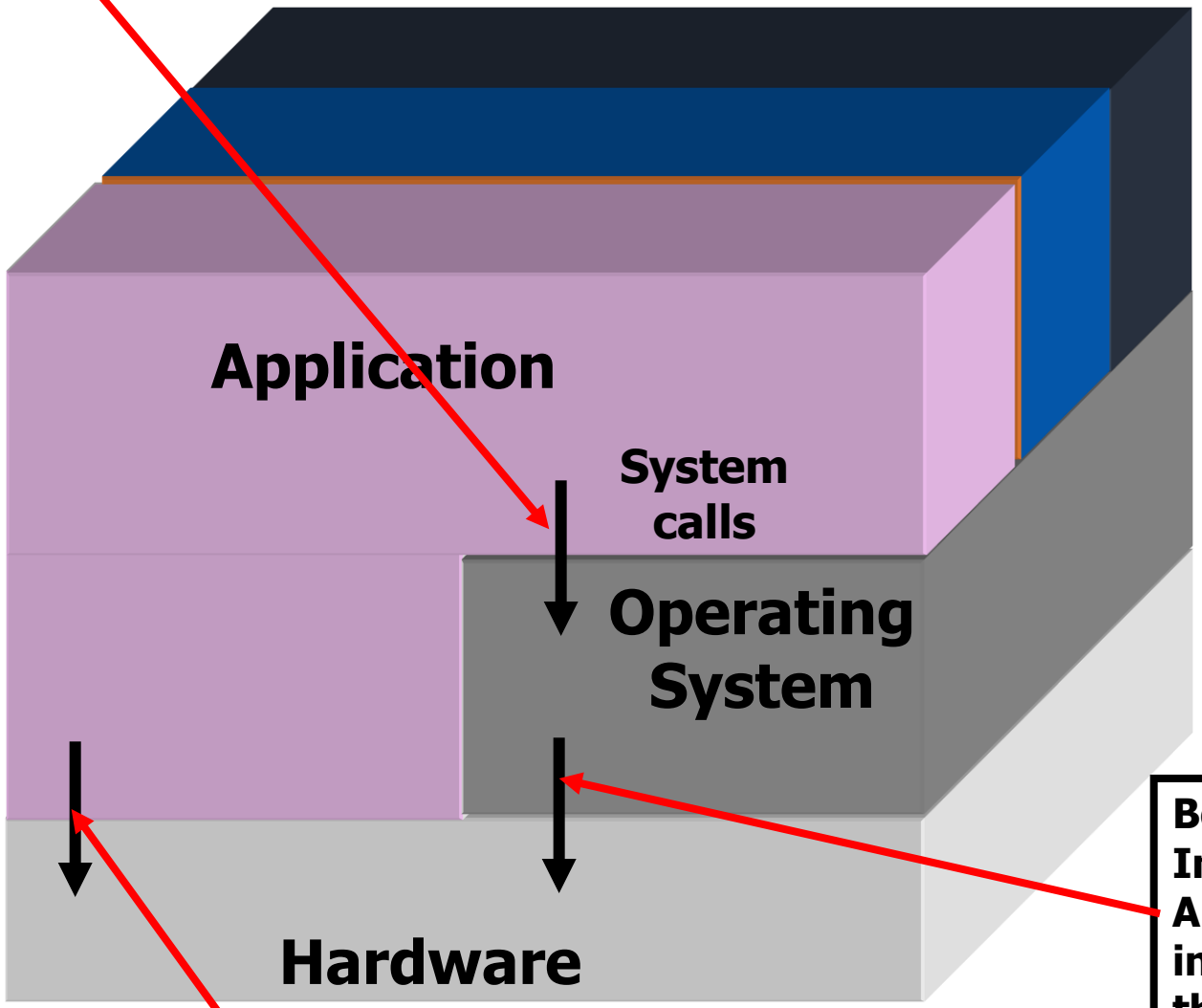
Introduction to Operating Systems

השקפים המסומנים ב- מהווים חומר הרחבה.

התוכן המובא בשקפים אלו <u>אינו</u> מהווה חלק מהחומר המחייב לבחינה בקורס.

The mechanism by which a process request the OS to perform a service for him. For example: open a file

מערכת ההפעלה:



Many applications

One Operating System

One Hardware

הציור בשקף זה מציג תיאור נוסף של יחסי הגומלין בין אפליקציות, לבין מערכת ההפעלה לבין החומרה.

הערה: תיאור מדויק יותר במובנים מסוימים מהתיאור המובא בשקף הקודם

Both privileged and non privileged Instructions
An example of a privileged instruction: HALT (which causes the CPU to stop working)

Non privileged instructions <u>only</u>. For example: ADD.

מה זו מערכת הפעלה ?

לא ניתן להגדיר 'ישירות' [= הגדרה מילונית] מה זו מערכת הפעלה.
למרות שבפועל קיימות הגדרות כאלו {בספרים, מסמכים טכניים ומקורות אחרים} – אף אחת מההגדרות האלו אינה מדויקת.
ויתירה מכך: אין בה כדי לתאר מה היא מערכת ההפעלה.
הדרך הטובה ביותר לנסות ולתאר מהי מערכת הפעלה – על ידי תיאור מאפייניה העיקריים !!

List of characteristics

1. The OS is a program
- The OS written in a high–level language
- The OS is a very complex program
- The OS is, almost, the first program and the last program that runs in the computer

2. The OS provides an abstraction of the ‘real world’

3. The OS is a resource manager (of the HW resources, as well as specific OS resources)

4. The OS provides services (via a system call’s API)

5. The OS provides protection and security (to the computer system as a whole and to programs)

List of characteristics

6. The term ‘an operating system’ refers to many different parts, among which
   - the OS kernel is considered to be the most ‘important’ [fundamental] part.
   - another important part is the OS {end} UI [User Interface]

7. The operating system is a reactive program (or: the OS is interrupt-driven)

8. The OS {tightly} ‘collaborates’ with the hardware architecture.
   - the hardware architecture MUST provide various mechanisms specifically designed to be used by the operating system
   - these mechanisms are ESSENTIALS in order for the OS to perform its role [‘fulfill its promises’]

The OS is a Program

מערכת ההפעלה היא תוכנה

- מערכת ההפעלה היא תוכנה
- מיליונים רבים של שורות קוד

- רובו הגדול של הקוד הזה (למעלה מ- 90%) כתוב בשפת גבוהה,
- והשאר כתוב בשפת אסמבלר של המעבד המותקן במחשב עבורו מיועדת מערכת ההפעלה.

מערכת ההפעלה Unix – נקודת ציון משמעותית בהיסטוריה של מערכות הפעלה

- כאמור, רובו הגדול של הקוד של מע' ההפעלה כתוב בשפת גבוהה.

- לא תמיד זה היה כך.
- מערכות ההפעלה הראשונות היו כתובות רק באסמבלר !
- עד לפיתוח של מערכת ההפעלה Unix.
- פרויקט מונומנטלי שבוצע במעבדות Bell.

- שניים מהאנשים הבולטים בפרויקט של פיתוח Unix היו Ken Thompson ו- Dennis Ritchie
- שבין היתר, לצורך הפיתוח של Unix, 'המציאו' שפת C.

- מעבר לעובדה שהפיתוח של Unix נועד 'להוכיח' שניתן לפתח מערכת הפעלה בשפה גבוהה, ולא בשפת אסמבלר, הרעיונות שפותחו ומומשו בגרסאות הראשונות של Unix היוו, ועדיין מהווים עד היום, 'רעיונות יסוד' בפיתוח מערכות הפעלה.
- רעיונות אלו 'מצאו את דרכם' למערכת ההפעלה Windows ולזו של MacOs וכו'
- דוגמא בולטת נוספת: מערכת ההפעלה Linux
- ובהקשר זה הגרסאות השונות של Android

מערכת ההפעלה 'מותאמת' לארכיטקטורת מעבד מסוים

- כאמור, רובו הגדול של הקוד של מע' ההפעלה כתוב בשפת גבוהה, אבל חלקים מסוימים מהקוד כתובים {חייבים להיכתב} באסמבלר.

- מהתיאור הנ"ל עולה כי: תכנית מערכת ההפעלה 'מותאמת' למעבד מסוים.
- יותר מדויק: לארכיטקטורה של מעבד.
- לדוגמא: למעבדי IA-32

מערכת ההפעלה 'מותאמת' לארכיטקטורת מעבד מסוים

- מדוע מערכת הפעלה מותאמת לארכיטקטורת מעבדים ספציפית ?

- ראשית, בשל ההכרח לכתוב חלקים מסוימים מהקוד באסמבלר.
- אבל לא רק בשל הסיבה הזו – חלקים מרכזיים בקוד של מע' הפעלה חייבים להיות מותאמים למאפיינים ספציפיים של ארכיטקטורת המעבד:
  - מנגנון הטיפול בפסיקות שנתמך [= ממומש] ע"י המעבד
  - מנגנון ניהול ה- MM שנתמך ע"י המעבד
  - מאפיינים נוספים של ארכ' המעבד

מערכת ההפעלה 'מותאמת' לארכיטקטורת מעבד מסוים

לפיכך, קיימות גרסאות שונות של מע' ההפעלה לינוקס:
למעבדי אינטל המממשים ארכ' IA-32
למעבדי אינטל המממשים ארכ' 64 ביט
למעבדי ARM המממשים ארכ' 32ביט
למעבדי ARM המממשים ארכ' 64 ביט
וכו'

קיימות גרסאות שונות של מע' ההפעלה Windows:
למעבדי אינטל המממשים ארכ' IA-32
למעבדי אינטל המממשים ארכ' 64 ביט

הקוד של מערכת ההפעלה הוא הקוד הראשון, כמעט, והאחרון שמבוצע על ידי המעבד

הקוד של מערכת ההפעלה הוא הקוד הראשון המתבצע ע"י המעבד

- הקוד של מערכת ההפעלה הוא הקוד [= התוכנה] הראשון המתבצע ע"י המעבד.
- מיד אחרי הפעלת המחשב [= power on]

- יותר מדויק: הקוד של מערכת ההפעלה הוא 'כמעט' קוד הראשון המתבצע.
- הקוד הראשון המתבצע הוא קוד שמממש את ה- **boot sequence** – סדרת צעדים שמטרתה:
  - לבצע בדיקות ראשוניות של החומרה,
  - לבצע אתחולים ראשונים של החומרה,
  - ולהעתיק את הקוד של מע' ההפעלה ל- MM מהתקן זיכרון לא נדיף {למשל HD, או SSD}
  - על מנת שהמעבד יבצע קוד זה

CPU modes

CPU modes

- מרגע ה- power on ועד רגע ביצוע HALT, המעבד מבצע שוב ושוב סדרת צעדים שקרויה **מחזור ה-הוראה [instruction cycle]**
- כל מחזור הוראה כולל 3 שלבים:
  - IF [Instruction Fetch]
  - ID [Instruction Decode]
  - IE [Instruction Execute]

- הארכיטקטורה של המעבד כוללת תמיכה במספר מצבים שנקראים CPU modes
- בשלב הזה של התיאור נתאר רק 2 מצבים
  - **מצב משתמש [user mode]**
  - **מצב גרעין [kernel mode]**
- בד"כ, במעבדים המודרניים, התמיכה היא ביותר מ- 2 מצבים

CPU modes & privileged/non privileged instructions

ההבדל בין המצבים האלו נוגע לשאלה 'אלו הוראת המעבד מוכן לבצע'
לדוגמא, במצב גרעין המעבד יבצע כל הוראה שהיא
בעוד שבמצב משתמש הוא יהיה מוכן לבצע' רק הוראות מסוימות: **הוראות לא מיוחסות**
[non privileged instruction]

- את קבוצת ההוראות ב- ISA [Instruction Set Architecture] ניתן לחלק ל- 2 תתי קבוצות:
  - הוראות לא מיוחסות **[non privileged instruction]**
    - דוגמאות: כל ההוראות האריתמטיות והלוגיות, הוראות data transfer
  - הוראות מיוחסות **[privileged instruction]**
    - דוגמאות: HALT, הוראות לשינוי ערכם של ביטים מסוימים ב- **PSW [Processor Status Word]**

אוגר ה- PSW

- כל מעבד כולל אוגר שנקרא **PSW [Processor Status Word]**
- הביטים השונים באוגר זה מייצגים את 'מצבו הנוכחי' של המעבד
- לדוגמא, אחד הביטים מציין האם התוצאה של הפעולה האריתמטית האחרונה שביצע המעבד הייתה שלילית

- מספר ביטים מציין את ה- CPU mode הנוכחי.
- בפועל מס' הביטים האלו מתאים למספר ה- CPU modes הנתמכים בארכיטקטורה
- למשל, אם ארכ' ARM תומכת ב- 7 מצבים, הרי שדרושים {לפחות} 3 ביטים כדי לציין CPU mode

- היות ואנחנו הנחנו 2 מצבים בלבד, הרי שדי בביט אחד בלבד לציין את ה- CPU mode הנוכחי.
- נקרא לביט הזה kernel bit
- ונניח כי אם ערכו של הביט הוא 1 אזי הדבר מציין שהמעבד נמצא ב- kernel mode
- וכמובן אם ערכו של הביט הוא 0 אזי הדבר מציין שהמעבד נמצא ב- user mode

privileged instructions & the kernel bit

- המעבד מזהה, בשלב ה- ID, את ההוראה שהוא מונחה לבצע
- אם מדובר בהוראה לא מיוחסת, אזי המעבד יבצע אותה [בשלב הבא, שלב ה- IE] בכל מקרה לא משנה מהו ערכו הנוכחי של ה- kernel bit

- אם, לעומת זאת, מדובר בהוראה מיוחסת, אזי המעבד יבצע אותה [בשלב הבא, שלב ה- IE] אך ורק אם ערכו הנוכחי של ה- kernel bit הוא 1
- אם ערכו הנוכחי של ה- kernel bit הוא 0, היינו המעבד נמצא במצב user, המעבד לא יבצע את ההוראה [= לא יבצע את שלב ה- IE]
- במקום זה תיווצר פסיקה אשר הטיפול בה יגרום להפסקת המשך הריצה של התהליך !!!
- נסיון ביצוע הוראה מיוחסת ע"י תהליך 'לא מורשה' [תהליך user]

איך זה קשור לענייננו בשלב הזה ?

- כאשר המעבד מתחיל לפעול, הוא באופן default-י נמצא במצב kernel
- היינו, ערכו של ה- kernel bit הוא 1
- לכן, במשך כל שלב ה- bootstrap המעבד נמצא במצב גרעין
- היינו, כל ההוראות שמתבצעות במהלרה- bootstrap נמצאות במצב גרעין

- אחרי שמסתיים שלב ה- bootstrap הקוד של מערכת ההפעלה והנתונים שלה 'נמצאים' ב- MM
- לפחות חלק מהקוד וחלק מהנתונים
  - בהמשך הקורס כאשר נדבר על זיכרון מדומה [memory] נבין למה הניסוח הוא: 'לפחות חלק מהקוד וחלק מהנתונים'

- ואז בשלב הזה, המעבד עובר לבצע את הקוד של מערכת ההפעלה
- בהתחלה הוא מבצע הוראות שמאתחלות מבני נתונים, ערכי משתנים וכו' של מע' ההפעלה
  - פרט לאתחולים אלו מתבצעים אתחולים נוספים
    - כמו למשל 'יצירה' של תהליך ראשוני שלצורך הדיון נקרא לו init.
    - יוסבר בהמשך

איך זה קשור לענייננו בשלב הזה ? (המשך)

- ז"א, כאשר המעבד מבצע את הקוד של מ"ה הוא נמצא במצב גרעין !!
- ולכן הקוד של מ"ה יכול לכלול כל הוראת מכונה שהיא !
- הוא יכול למשל, לחסום פסיקות [הוראה מיוחסת]
- לשנות את ה- CPU mode למצב משתמש [= לשנות את ערכו של ה- kernel bit ל- 0]
- גם זו כאמור הוראה מיוחסת
- מ"ה תבצע הוראה זו לפני שהיא 'מעבירה' [מקצה] את המעבד לטובת תכנית אחרת [תכנית משתמש]
- יוסבר בהמשך הקורס
- תכנית 'רגילה' לא יכולה לשנות את ערכו של ה- kernel bit !
- בהקשר לנאמר לעיל, עולה השאלה, איך 'דואגים' לכך שערכו של ה- kernel bit המעבד ?
- תשובה: באמצעות מנגנון הפסיקות הנתמך ע"י המעבד
- יוסבר בהמשך

הקוד של מערכת ההפעלה הוא הקוד האחרון המתבצע ע"י המעבד

- הקוד של מערכת ההפעלה הוא הקוד האחרון המתבצע ע"י המעבד
- הקוד שמבצע את ה- **shutdown sequence**
- שההוראה האחרונה בו היא ההוראה HALT
- הוראה מיוחסת
- שגורמת 'לעצירה מוחלטת' של המעבד [= הפסקת ביצוע חוזר ונשנה של **מחזור הוראה** (= **instruction cycle**)]

אבסטרקציה של 'העולם האמיתי'
[= של מה 'שקורה באמת']

מערכת ההפעלה יוצרת אבסטרקציה

- אחד התפקידים המרכזיים של מע' ההפעלה הוא ליצור אבסטרקציה
- מה פירוש ? להציג תמונת עולם נוחה יותר מתמונת העולם 'האמיתי' [= ממה שבאמת קורה במחשב]

- תמונת עולם אבסטרקטית זו מיועדת כדי 'להפוך' את החומרה לברת שימוש
- ללא מע' ההפעלה השימוש בחומרה [= הניצול של החומרה] לצורך הרצת קוד, לצורך שמירת מידע, לצורך החלפת מידע בין מחשבים וכו' – מאוד מסובך.
- מה שיגרום לכך שכל מנגנוני החומרה המורכבים והמסובכים יהיו ברי שימוש רק לקומץ של אנשים.
- את העובדה כי רוב האנשים היום משתמשים במחשב ניתן בין היתר ל'זקוף לזכותה' של מערכת ההפעלה.

הערה #1

- את העובדה כי רוב האנשים היום משתמשים במחשב ניתן בין היתר ל"זקוף לזכותה' של מערכת ההפעלה.
- לא רק לזכותה של מע' ההפעלה, אלא כמובן גם ל"גורמים' נוספים:
  - קומפיילרים [שמתרגמים משפת תכנות גבוהה לשפת אסמבלר]
  - תכניות assembler [= אותן התכניות שמתרגמות משפת אסמבלר לשפת מכונה]
  - ספריות
  - וכו'

אבל ללא ספק מערכות ההפעלה תרמו, ותורמות, משמעותית לשימוש הנפוץ היום במחשבים.

הערה #2

רוב האנשים היום משתמשים במחשב:

- ישירות – מחשבים שולחניים, לאפטופים, טאבלטים, סמארטפונים וכו'

- ב'עקיפין'

- ראוטרים ביתיים

- ממירים,

- קונסולות משחקים

- אוזניות 'אקטיביות'

- כלי רכב, מכונות כביסה, מכשירי מיקרוגל

- ועוד ועוד

אבסטרקציה למי ?

כאמור, אחד התפקידים המרכזיים של מע' ההפעלה הוא ליצור אבסטרקציה של העולם האמיתי, וע"י כך להפוך את השימוש הישיר במחשבים לקל יחסית, ופועל יוצא מכך לנגיש.

למי האבסטרקציה מיועדת ?

- משתמשי קצה
  - דרך [= באמצעות] ממשק המשתמש הגרפי
    - שמגיע עם כל מערכת ההפעלה
- מתכנתים
  - מתכנתים 'רגילים'
  - system programmers
    - כאלו שמשלבים בקוד שהם מפתחים קריאות לקבלת שירותים של מערכת ההפעלה
      - **קריאות מערכת [system calls]**
        - לדוגמא: קריאות לפתיחת קובץ
          - למשל, קריאה ל- open() בלינוקס. <u>ולא</u> הקריאה ל- fopen()
        - או, דוגמא אחרת: קריאות ליצירת socket: socket()
          - <u>ולא</u> יצירת סוקט דרך {הספריות של} Python

אבסטרקציה איך?
ע"י יצירת וניהול אובייקטים

יצירת וניהול אובייקטים

כמור לעיל, מערכת ההפעלה היא התוכנה שרצה מהדלקת המחשב ועד כיבויו.

במהלך פרק הזמן הזה מע' ההפעלה יוצרת ומנהלת אובייקטים:
- תהליכים ו- threads
- קבצים

- כמו במקרה של כל אובייקט מדובר באבסטרקציה של איזושהי מציאות 'אמיתית' שהיא הרבה יותר מורכבת ומסובכת להבנה.
- וזאת משום שהבנת המציאות הזו מחיבת היכרות טובה מאוד של
  - הרבה פרטים טכניים מאוד שהם מאוד low level
  - הקשר בין פרטים אלו

ניהול אובייקטים ע"י רשומות ומבני נתונים

- לכל אובייקט כזה יש ייצוג פנימי בתכנית מע' ההפעלה: ע"י רשומות [structs]
  - רשומת תהליך – PCB {task struct בלינוקס}
  - רשומת thread – TCB {task struct בלינוקס}
  - רשומת קובץ – {רשומת inode בלינוקס}

- בנוסף, כחלק מיצירת האבסטרקציה, מע' ההפעלה מנהלת אוסף של מבני נתונים:
  - טבלת תהליכים [process table]
  - רשימה מקושרת של threads
  - תורים [queues =]
    - תור המתנה למעבד [ready queue ובקצרה: readyQ]
    - תור המתנה לקלט
    - ...
  - טבלת קבצים

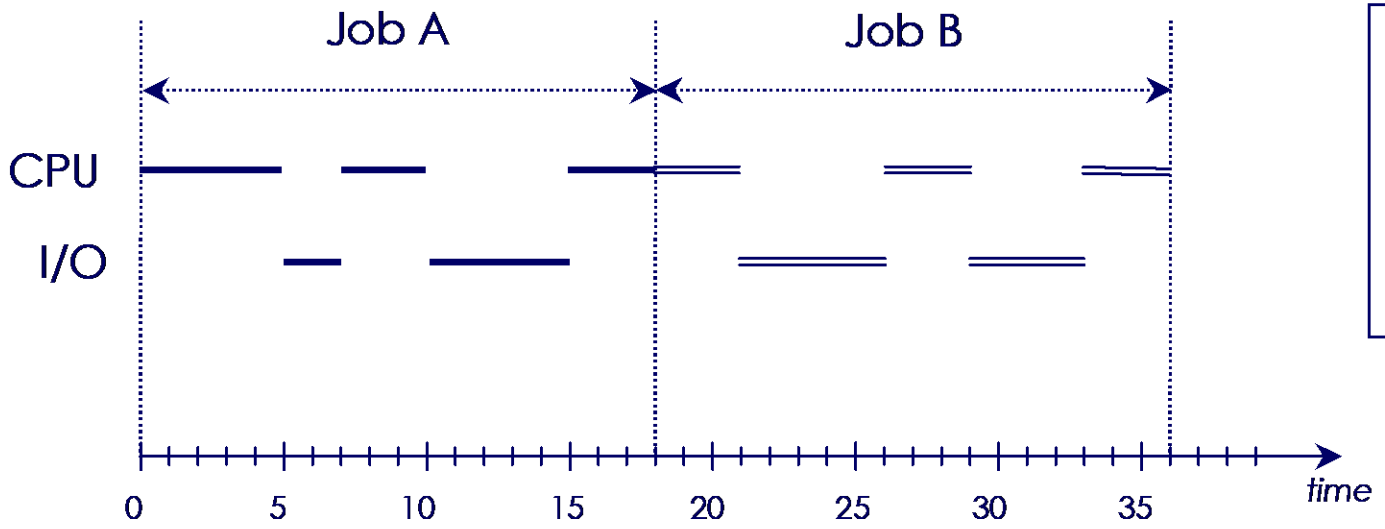
פעולות על אובייקטים

- מע' ההפעלה 'מספקת' [= מממשת] פעולות על האוביקט
  - יצירת אובייקט חדש,
  - יצירת תהליך, יצירת Thread
  - מחיקת אובייקט קיים

- פעולות ספציפיות לאובייקט מסוג מסוים:
  - דוגמאות לפעולות על אובייקט מסוג 'תהליך':
    - שליחת סיגנל לתהליך,
    - סנכרון בין תהליכים,
    - שיתוף אזור זיכרון MM בין תהליכים [shared memory]
  - דוגמאות לפעולות על אובייקט מסוג 'קובץ':
    - קריאה מקובץ,
    - כתיבה לקובץ,
    - שינוי מיקום סמן קובץ,
    - שינוי הרשאות גישה לקובץ,
    - וכו'

Resource management

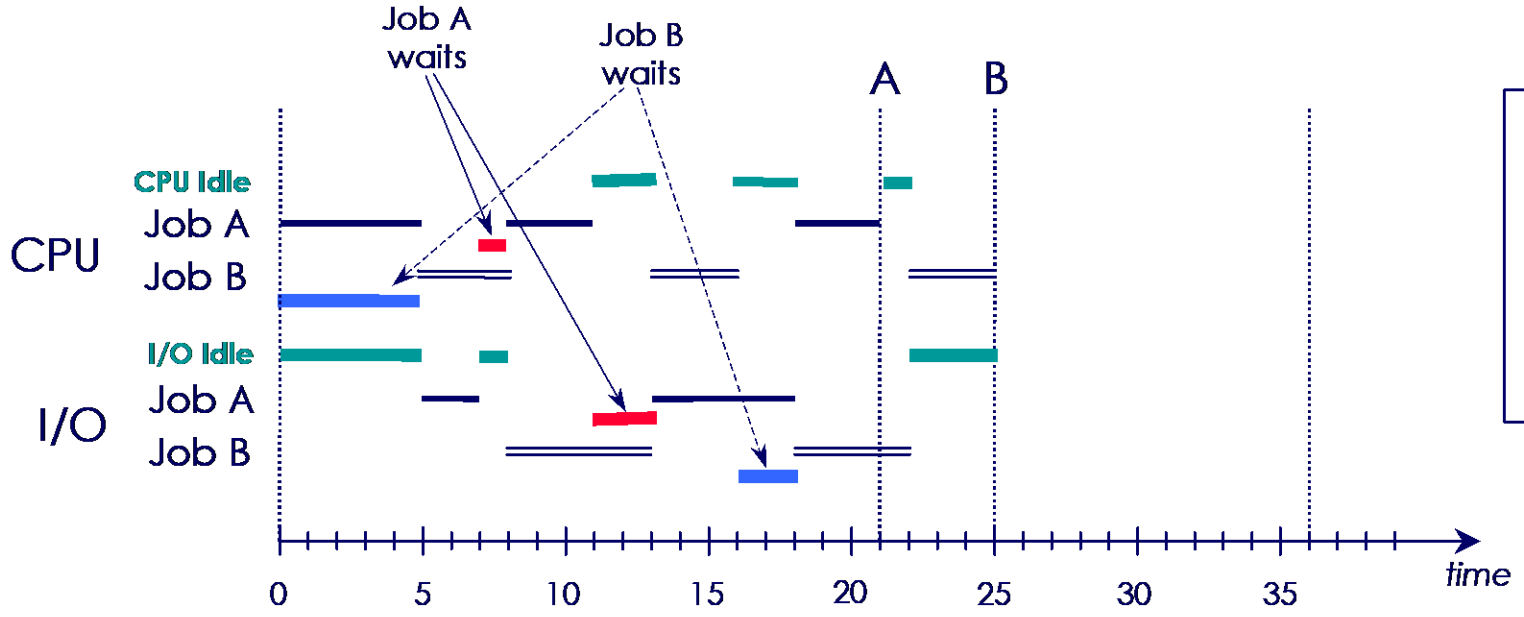
Multiprogramming support



uniprogramming

Jobs A and B end at time 36.

CPU is idle 47%
I/O is idle 53%



"pure" multiprogramming

Jobs A and B end at time 25.

CPU is idle 20%
I/O is idle 36%

ניהול משאבים

אלו משאבים ?

- מ"ה מנהלת:
  - את משאבי החומרה
  - משאבים ייחודיים שלה

ניהול משאבי החומרה

- מע' ההפעלה מחליטה לאיזו תכנית רצה [= תהליך] היא מקצה זמן מעבד, לפי איזו מדיניות [policy], וכו'
- כמו כן היא מנהלת את ה- MM – מחליטה כמה MM פנוי להקצות [ואת 'השאר' ב- VM]
- מנהלת את מרחב זיכרון האחסון.
- למשל במקרה של דיסק קשיח: מנהלת את בלוקי הנתונים של הדיסק

בהמשך נבין יותר טוב ויותר מדויק מה הכוונה 'מנהלת את משאבי החומרה'

ניהול משאבים 'יחודיים'

- מנהלת משאבים ייחודיים שלה:
- File descriptors
- Sockets
- מנעולים למיניהם: locks, semaphores, mutex
- ועוד !!

ניהול משאבי החומרה

מערכת ההפעלה כמנהלת משאבי החומרה

- משאבי החומרה כוללים:
  - מעבד/ים [ליבה/ליבות]
  - זיכרון ראשי (MM)
    - זיכרון נדיף. טכנולוגיית DRAM. ממומש ע"י טרנזיסטורים {semiconductor memory}
  - זיכרון משני
    - זיכרון לא נדיף [NVM = Non Volatile Memory]
  - וכו'

מערכת ההפעלה כמנהלת משאבי החומרה

- ש: מה פירוש שמערכת ההפעלה משמשת כמנהלת משאבי החומרה [= resource manager] ?
- ת: מערכת ההפעלה:
  - מקצה את משאבי החומרה לתהליכים ול- threads השונים שהיא מנהלת ברגע מסוים
    - היינו, תהליכים ו- threads אשר נוצרו ועדיין קיימים [= לא הסתיימו]
  - במקרים הרלבנטיים מתזמנת (schedule) את השימוש במשאבים.

- באופן יותר מפורט:
  - מקצה לכל תהליך מקום בזיכרון הראשי

  - מקצה לכל תהליך בלוקים בדיסק הקשיח
    - לבקשתו. בעקבות בקשה לשמירת מידע בקובץ.
    - לא תמיד. רק כאשר יש הכרח להגדיל את מספר הבלוקים שכבר מוקצים לתהליך

  - מקצה את המעבד ל- thread מסוים
    - ל- thread אשר ממתין להקצאת מעבד כדי שהמעבד 'יריץ' אותו'
    - היינו, כדי שהמעבד יבצע הוראות המכונה ה'משויכות' ל- thread
  - במידה ויש מספר threads שממתינים להקצאת המעבד – מתזמנת ביניהם

MM Management

ניהול ה- MM

- תהליך = תוכנית בריצה = תכנית שהמעבד מריץ = תכנית שהמעבד מבצע את הוראות המכונה שלה
- ז"א, התכנית חייבת להימצא ב- MM:

א. הוראות המכונה של התכנית

ב. המקום שהוקצה עבור המשתנים הגלובליים של התכנית

ג. מקום שמנוהל כמחסנית ומיועד עבור:

- המשתנים הלוקאליים
- עבור פרמטרים {נתונים} המועברים לפונקציות
- כאשר פונקציה f קוראת לפונקציה g, אזי הפרמטרים איתם f קוראת ל- g [= מעבירה ל- g] מועברים ל- g 'דרך' האזור המנוהל כמחסנית

ד. מקום שמנוהל כ- heap ומיועד לשרת בקשות הקצאת זיכרון דינמיות

- בקשות להקצאת זיכרון שמתבצעות תוך כדי ריצה.
- ע"י, למשל, קריאת ל- malloc()

- מי מקצה את ה- MM ? ה- OS
- כאשר מריצים תכנית, אזי ה- OS <u>טוענת עותק של קובץ</u> ההרצה ל- MM
  - התכנית בשפת מכונה והנתונים של התכנית נטענים לזיכרון

ניהול ה- MM

- מי מקצה את ה- MM ? ה- OS
- ה- OS מנהלת רישום אלו אזורי MM פנויים [= לא מוקצים]
- וכאשר ה- OS מקבלת בקשה להרצת תכנית [= יצירת תהליך חדש] היא 'בודקת' אלו אזורי MM פנויים ומקצה מהם

- **מנהל הזיכרון** [= **memory manager**]:
- החלק במע' ההפעלה [= הקוד = אוסף של פונקציות] שאחראי ל:
  - ניהול הרישום על אזורי ה- MM הפנויים
  - הקצאת אזורים פנויים
    - אזור ה- MM המוקצה לתהליך <u>אינו רציף</u> [= מספר אזורים לא רציפים]
      - היינו, אזור ה- MM המוקצה לתהליך הוא אוסף של
        - **סגמנטים**
        - או: **דפים** [**pages**]
    - במקרה של 'תכניות גדולות' [= הרבה הוראות מכונה ו/או הרבה מקום הדרוש עבור המשתנים הגלובליים, ה- OS לא מקצה 'כמות' MM שמספיקה לשמור את כל הוראות המכונה ומספיקה עבור כל המשתנים הגלובליים.
      - רק 'חלק' מהמקום
      - שאר המקום הדרוש מוקצה בזיכרון המשני
        - **זיכרון מדומה** [= **Virtual Memory**]

ניהול ה- MM

- לאחר ההקצאה של אזורי MM אלו לתהליך חדש, ה- OS:
- מעתיקה את הוראות המכונה, או חלק מהן, מקובץ ה- executable לאזורי ה- MM 'המתאימים'
- מעתיקה את ערכם ההתחלתי של המשתנים הגלובליים המאותחלים לאזורי ה- MM 'המתאימים

Processor management

ניהול המעבד

- תהליך = תוכנית בריצה = תכנית שהמעבד מריץ = תכנית שהמעבד מבצע את הוראות המכונה שלה

- נניח לצורך פשטות הדיון בשלב הזה כי התכנית היא single threaded
- ותחת הנחה זו: לצורך 'הרצת תכנית' יש להקצות את המעבד ל- thread
= 'לגרום' למעבד לבצע הוראות מכונה שמשויכות ל- thread
= 'לגרום' למעבד לקרוא הוראות מאזור ה- MM בו שמורות הוראות המכונה שמשויכות ל- thread ולבצע אותן
- תזכורת: מחזור הוראה = IF + ID +IE
- IF [= Instruction Fetch]: הבאת הוראת מכונה השמורה ב- MM
- ID [= Instruction Decode]:תה- opcode של הוראת מכונה
- IE [= Instruction Execute]: ביצוע הפעולות אותן מנחה הוראת מכונה לבצע [פעולה אריתמטית, או פעולה של data transfer, או הסתעפות וכו']

= לשמור באוגר ה- PC כתובת ששייכת למרחב כתובות ה- MM שהוקצו ל- thread

ניהול המעבד

- מע' ההפעלה היא זו שמקצה את המעבד ל- thread
- היינו, היא זו ששומרת באוגר ה- PC כתובת ששייכת למרחב כתובות ה- MM שהוקצו ל- thread
- וע"י כך למעשה מע' ההפעלה 'מוותרת' על המעבד
- שכן שמירת כתובת המתאימה [= זו ששייכת למרחב כתובות ה- MM שהוקצו ל- thread] באוגר ה- PC נעשית הרי ע"י הוראת מכונה
- הוראת מכונה ש'שייכת' [= חלק מכלל הקוד של מע' ההפעלה]
- מה שאומר שברגע ההקצאה המעבד היה 'שייך' למע' ההפעלה [= ביצע את הקוד שלה]
- ומיד אחרי ההקצאה של המעבד הוא כבר לא יהיה 'שייך' למע' ההפעלה. אלא ל- thread שעבורו הוא הוקצה.
- היינו, בעצם הקצאת המעבד ל- thread, מ"ה 'מוותרת' על המעבד לטובת ה- thread

ניהול המעבד

- הקצאת המעבד ל- thread היא לא בהכרח ברצף.
- היינו לא בהכרח למשך כל פרק הזמן מרגע תחילת הריצה של ה- thread ועד רגע סיום הריצה

- מערכת ההפעלה 'לוקחת' את המעבד מה- thread:
  א. כאשר ה- thread ממתין להתרחשות של event מסוים
  - לדוגמא: לזמינות של קלט בהתקן הקלט הסטנדרטי
  - במקרה כזה, כל עוד ה- thread ממתין לקלט אין לו הרי צורך במעבד
  ב. מעת לעת

Storage memory management

ניהול זיכרון האחסון

- כאשר thread רץ, הוא יכול לקרוא מידע מקבצים ו/או לכתוב מידע לקבצים
- ז"א: קורא מידע מזיכרון האחסון, או כותב מידע לזיכרון זה.
- מערכת ההפעלה מנהלת את הקריאות ואת הכתיבות לזיכרון האחסון

The OS provides services to Processes and to Threads

מ"ה 'מספקת' שירותים לתהליכים ול- threads

- מ"ה מספקת שירותים {services} לתכניות הרצות [לתהליכים ול- threads]

שירותים – למה ?

- למה מ"ה מספקת שירותים {services} לתכניות הרצות [לתהליכים ול- threads] ?

1. כדי להקל על כתיבת תכניות
   - לדוגמא: ביצוע הצעדים הדרושים בפועל כדי לקרוא קלט מהתקן קלט
   - ביצוע הצעדים הדרושים בפועל כדי לשלוח מידע באמצעות כרטיס רשת { Ethernet, WiFi, Bluetooth, celluar וכו'}

2. כדי ל'הבטיח' ע"י כך הגנה
   - התהליכים וה- threads '<u>נאלצים</u>' ['מחויבים'] לבקש שירותים {מסוימים} ממ"ה
   - <u>הם אינם יכולים לממש בעצמם שירותים אלו</u>
   - וע"י האופן שבו מ"ה מממשת שירותים אלו, היא מבטיחה הגנה
     - על עצמה מפני תכניות אחרות
     - על תכניות אחת מפני השנייה

The OS system call's API

שירותים – איך ?

- איך תהליכים ו- threads 'מבקשים' שירותים {services} ממ"ה ?

- על ידי ביצוע **קריאות מערכת** [system calls]
- המונח 'קריאת מערכת' הוא קיצור של 'קריאה לקבלת שירות של מערכת ההפעלה'

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t     read(int fd, void *buf, size_t count)
```

return value — function name — parameters

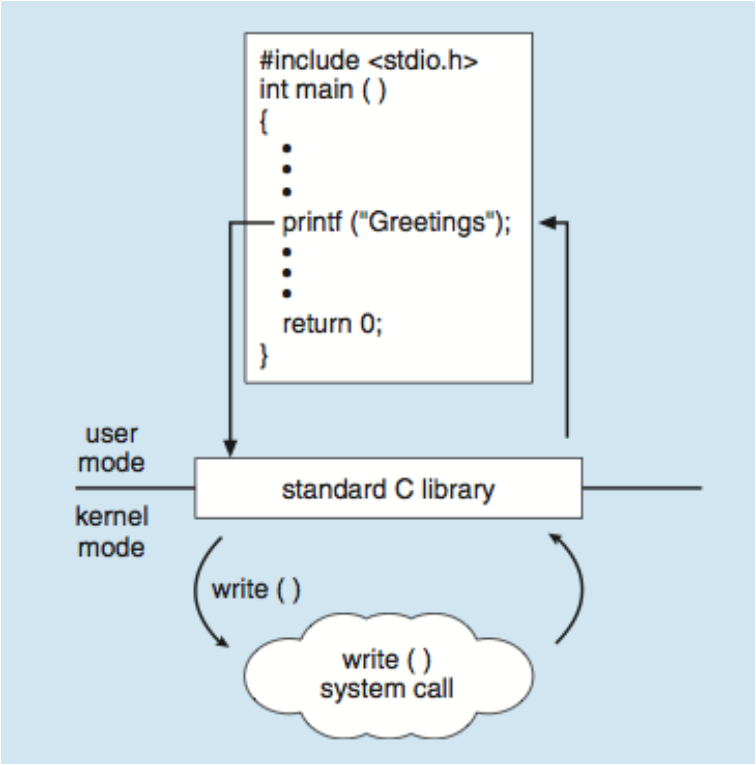
A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns −1.

Standard C Library Example

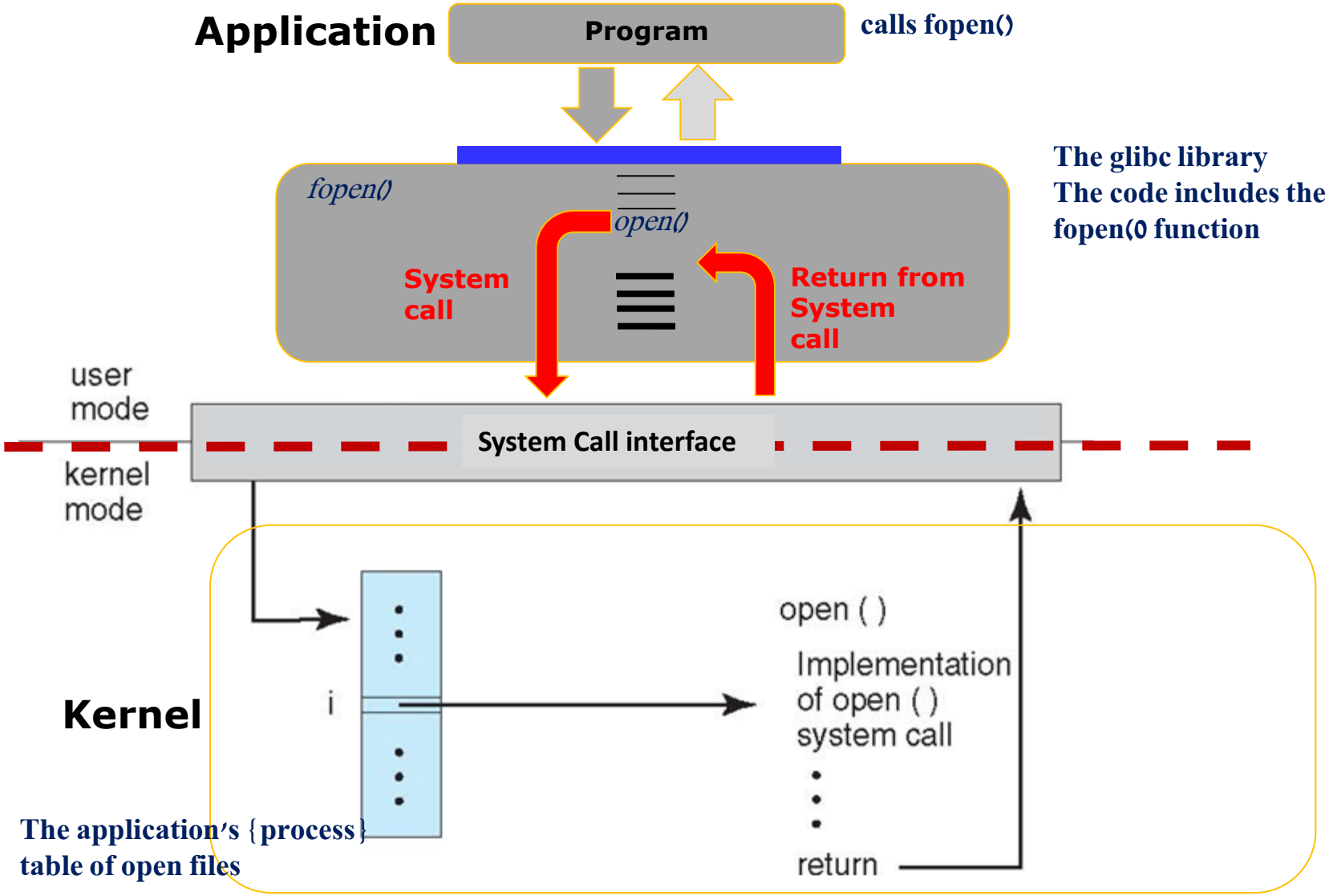
- C program invoking printf() library call, which calls write() system call



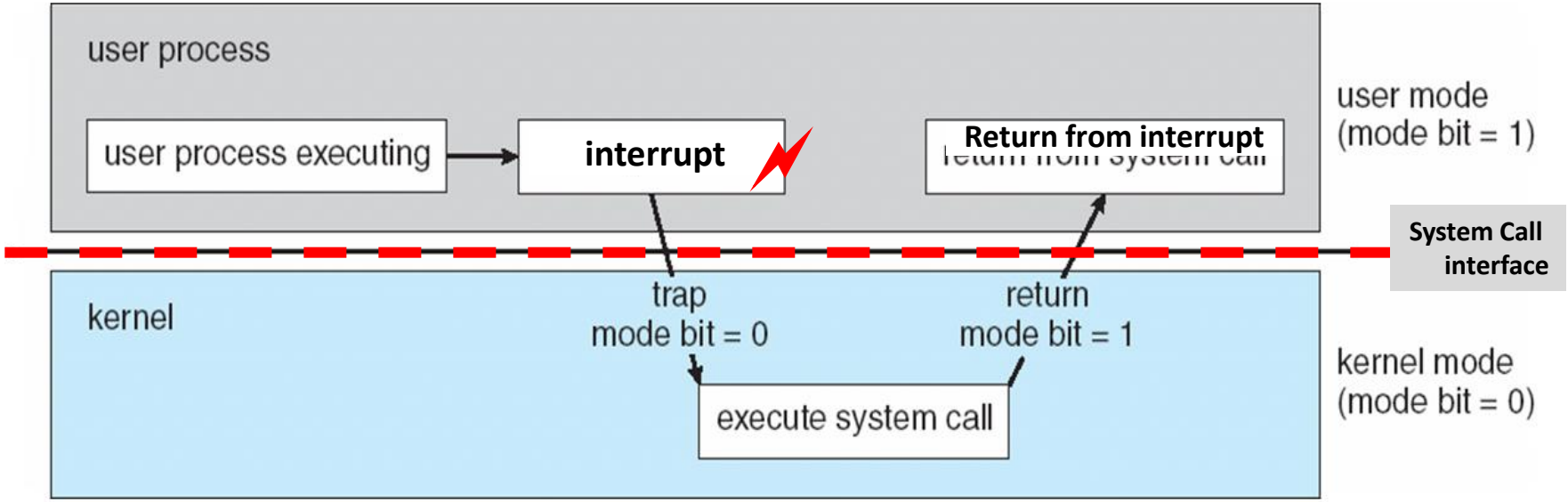
The primary function of the wrapper is to

- Place all the **arguments** to be passed to the system call in the appropriate processor registers (& maybe on the call stack as well)
- Set a unique **system call number** for the kernel to call. In this way the library, which exists between the OS and the application, increases portability.

system calls implementation


Application
Program
calls fopen()
fopen()
open()
System call
Return from System call
The glibc library
The code includes the code of the fopen(0 function
user mode
kernel mode
System Call interface
Kernel
i
open ( )
Implementation of open ( ) system call
return
The application's {process} table of open files

Kernel Mode and User Mode



Protection & security

הגנה [protection]

- מערכת ההפעלה מספקת הגנה:

1. על עצמה מפני תכניות אחרות
- אם מ"ה 'מושבתת' אזי כל מערכת המחשב מושבתת
- להבדיל מתכנית בודדת, שאם היא 'עפה' אזי שאר התכניות ממשיכות לרוץ

2. על תכניות אחת מפני השנייה

הגנה

3. על כלל מערכת המחשב
- מניעת 'השתלטות' (עוינת או שלא מכוונת) על משאבים חיוניים
- למשל על המעבד

הרצת תהליכי שמנטרים באופן שוטף את הפעילות במערכת המחשב במטרה לזהות:
פעילות חשודה
צריכה מוגברת של משאבים
זמן מעבד, נפח דיסק פנוי, וכו'

security

- הרצת תהליכים שמנטרים באופן שוטף את הפעילות במערכת המחשב במטרה לזהות:
  - פעילות חשודה
  - צריכה מוגברת של משאבים
  - זמן מעבד, נפח דיסק פנוי, וכו'

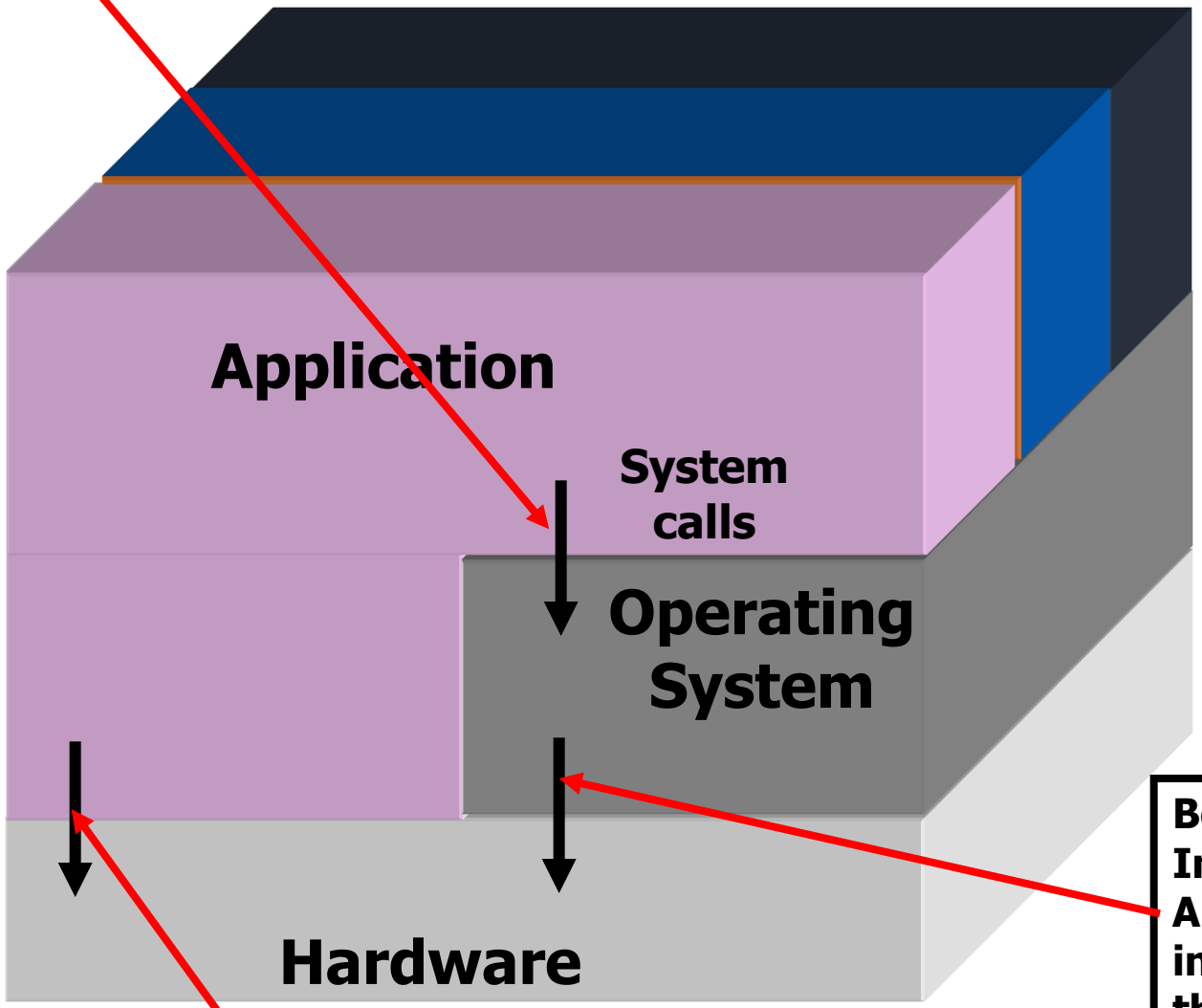
Operating system =
the OS kernel +
GUI + {many} other ‘parts’ [modules}

OS vs. OS kernel

- קיים הבדל בין המונח מערכת הפעלה [OS] לבין המונח **הגרעין** [**kernel** OS] של מערכת ההפעלה

- בשלב ההתחלתי הזה של הקורס, נאמר רק כי המונח OS מתייחס למכלול (אוסף) של מרכיבים, ובכלל זה:
  - הגרעין
  - תכנית ה- UI
  - ...
  - ספריות
  - קבצי הגדרה

The mechanism by which a process request the OS to perform a service for him. For example: open a file



Non privileged instructions <u>only</u>. For example: ADD.

Both privileged and non privileged Instructions
An example of a privileged instruction: HALT (which causes the CPU to stop working)

Many applications

One Operating System

One Hardware

מערכת ההפעלה:
<u>תיאור מדויק יותר</u>

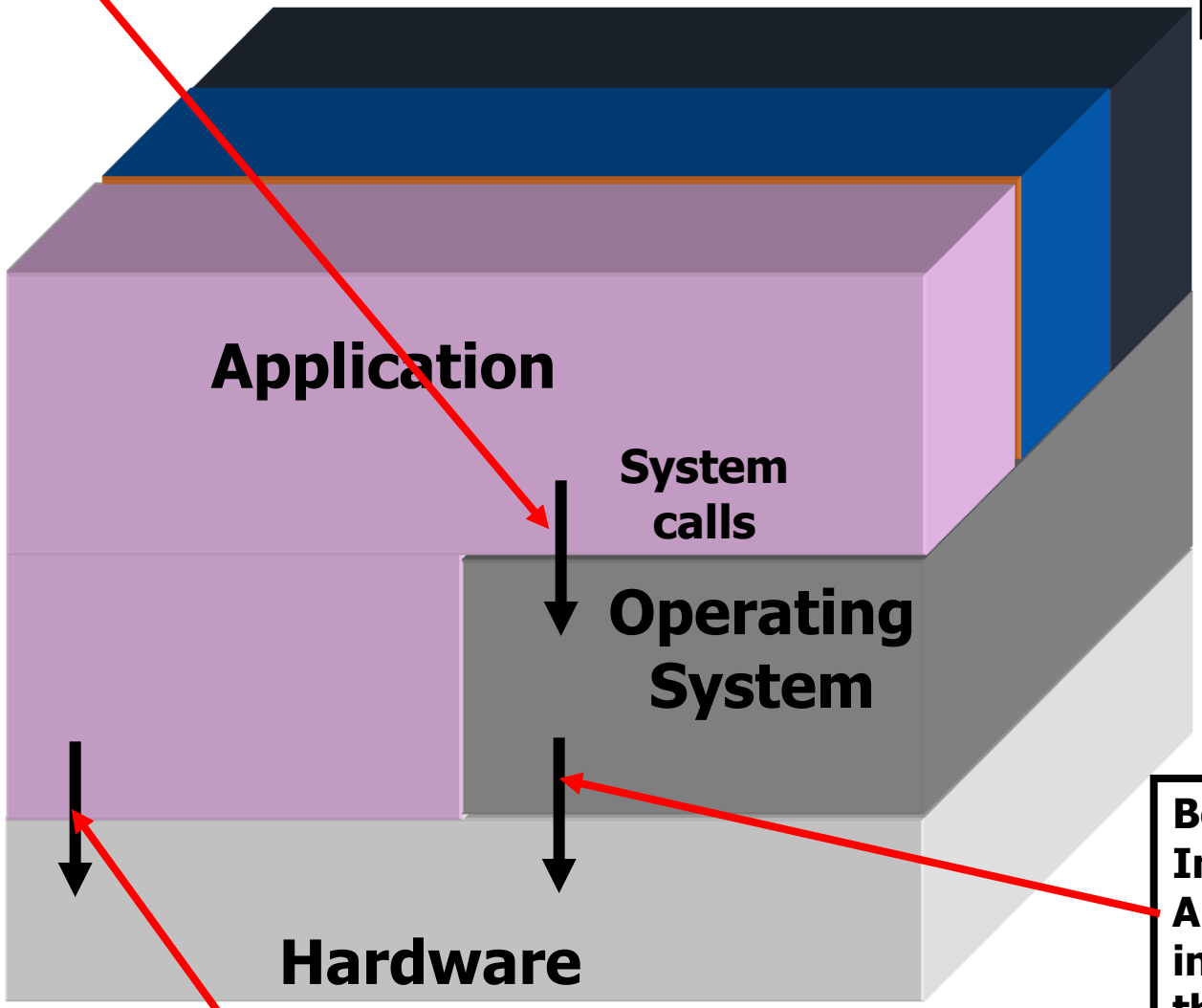
הציור בשקף זה מציג תיאור נוסף של יחסי הגומלין בין אפליקציות, לבין מערכת ההפעלה לבין החומרה.

הערה: תיאור מדויק יותר במובנים מסוימים מהתיאור המובא בשקף הקודם

לפי התיאור בציור, התוכנה במערכת המחשב מתחלקת ל- 2:

- תוכנת (קוד) הגרעין
- כל 'שאר' התוכנה [= כל שאר התוכניות + הספריות וכו']

The mechanism by which a process request the OS to perform a service for him. For example: open a file



Both privileged and non privileged Instructions
An example of a privileged instruction: HALT (which causes the CPU to stop working)

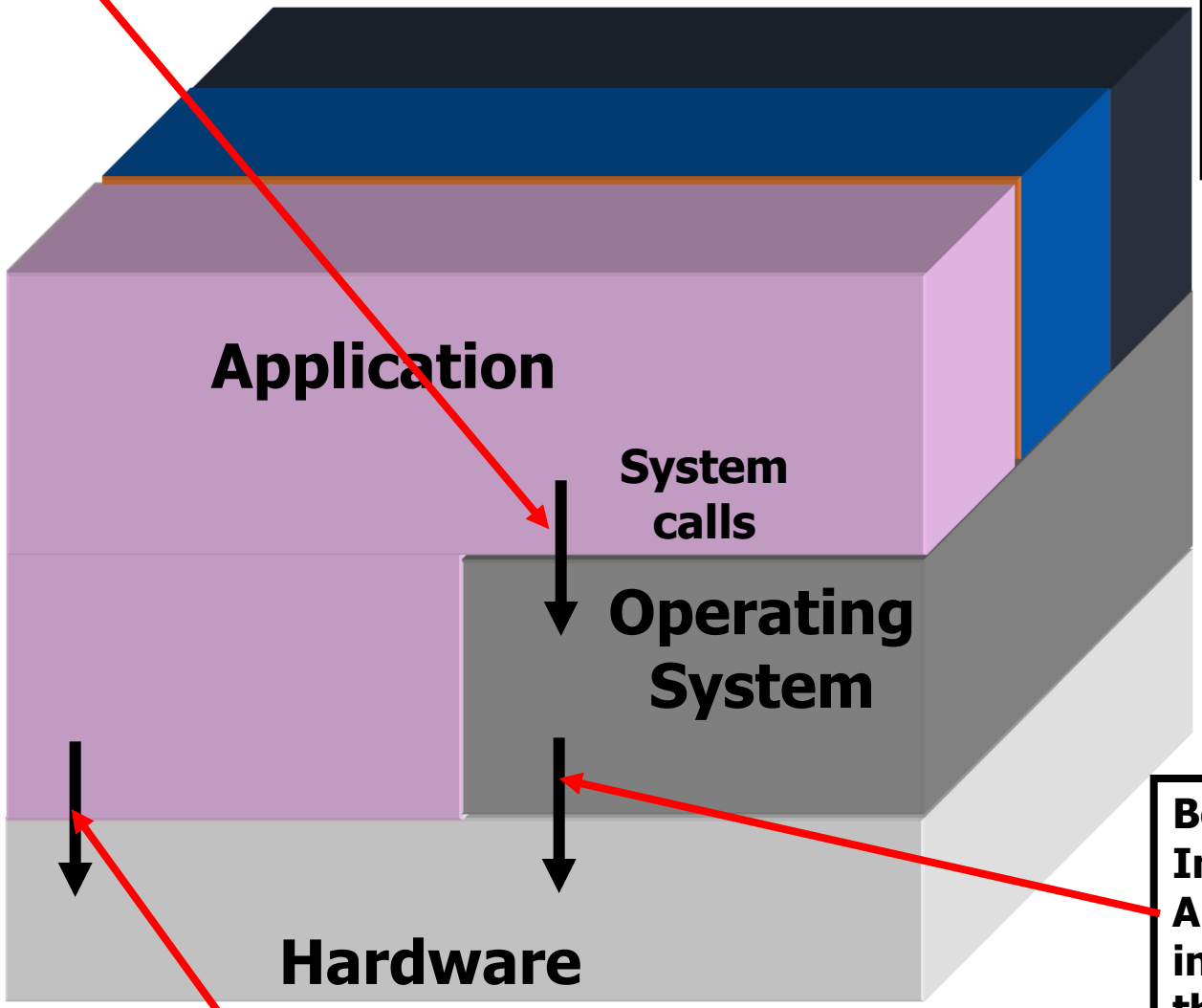
Non privileged instructions <u>only</u>. For example: ADD.

The mechanism by which a process request the OS to perform a service for him. For example: open a file

מתכנני ארכיטקטורת החומרה מתכננים את:

- מאפייני קבוצת ההוראות של המעבד (ה- ISA של המעבד)
- מצבי הריצה (privileged levels)
- מנגנון הפסיקות הנתמך (ממומש) ע"י המעבד

[ראו השקף בהמשך לפירוט נרחב יותר]



Both privileged and non privileged Instructions
An example of a privileged instruction: HALT (which causes the CPU to stop working)

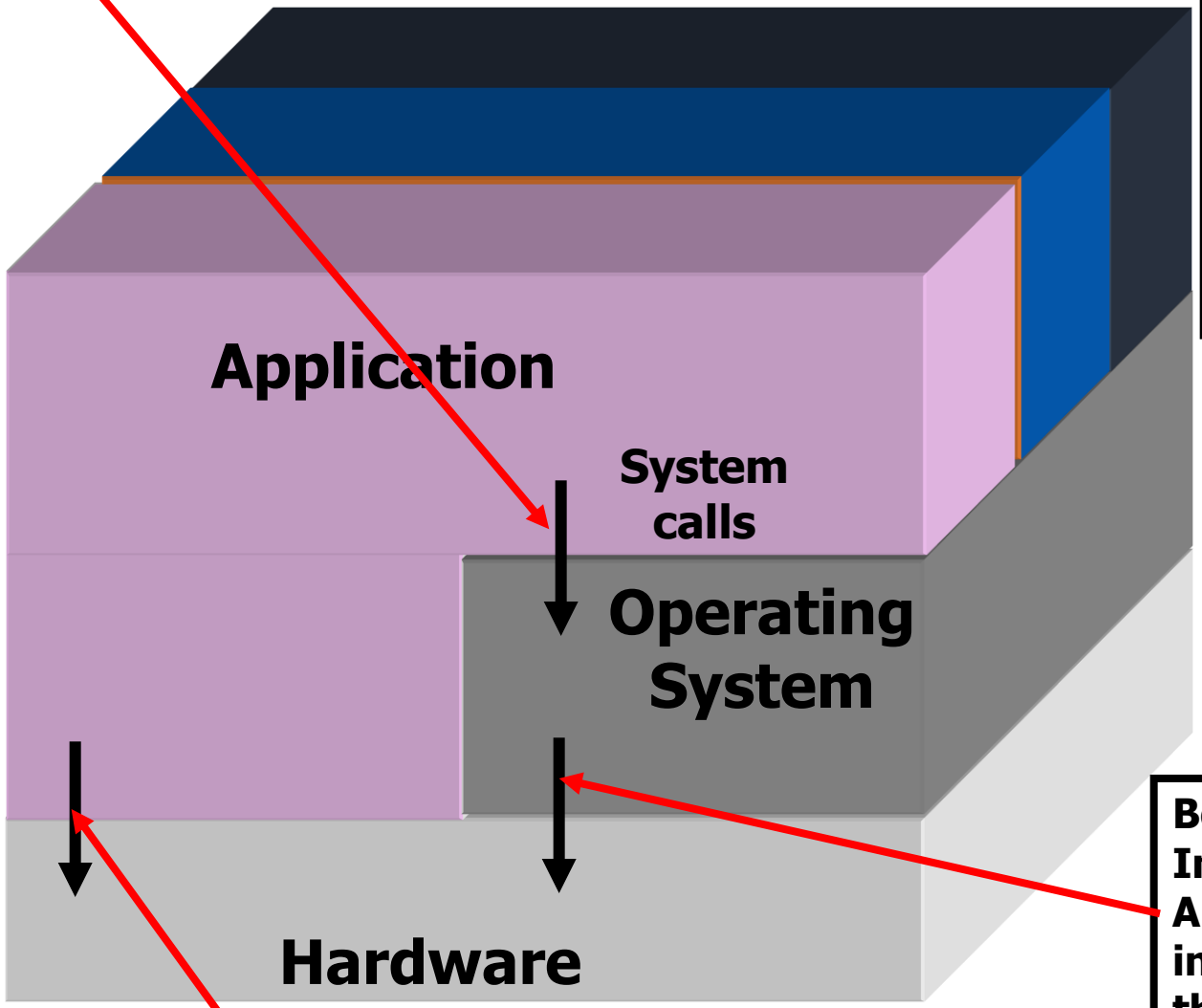
Non privileged instructions <u>only</u>. For example: ADD.

The mechanism by which a process request the OS to perform a service for him. For example: open a file

כחלק מהתכנון של ה- ISA, מתכנני ארכיטקטורת המעבד 'מחלקים' את כלל ההוראות ל- 2 קבוצות: הוראות מיוחסות + הוראות לא מיוחסות.

כחלק התכנון של מצבי הריצה (privileged levels), הם מגדירים לפחות 2 מצבים:

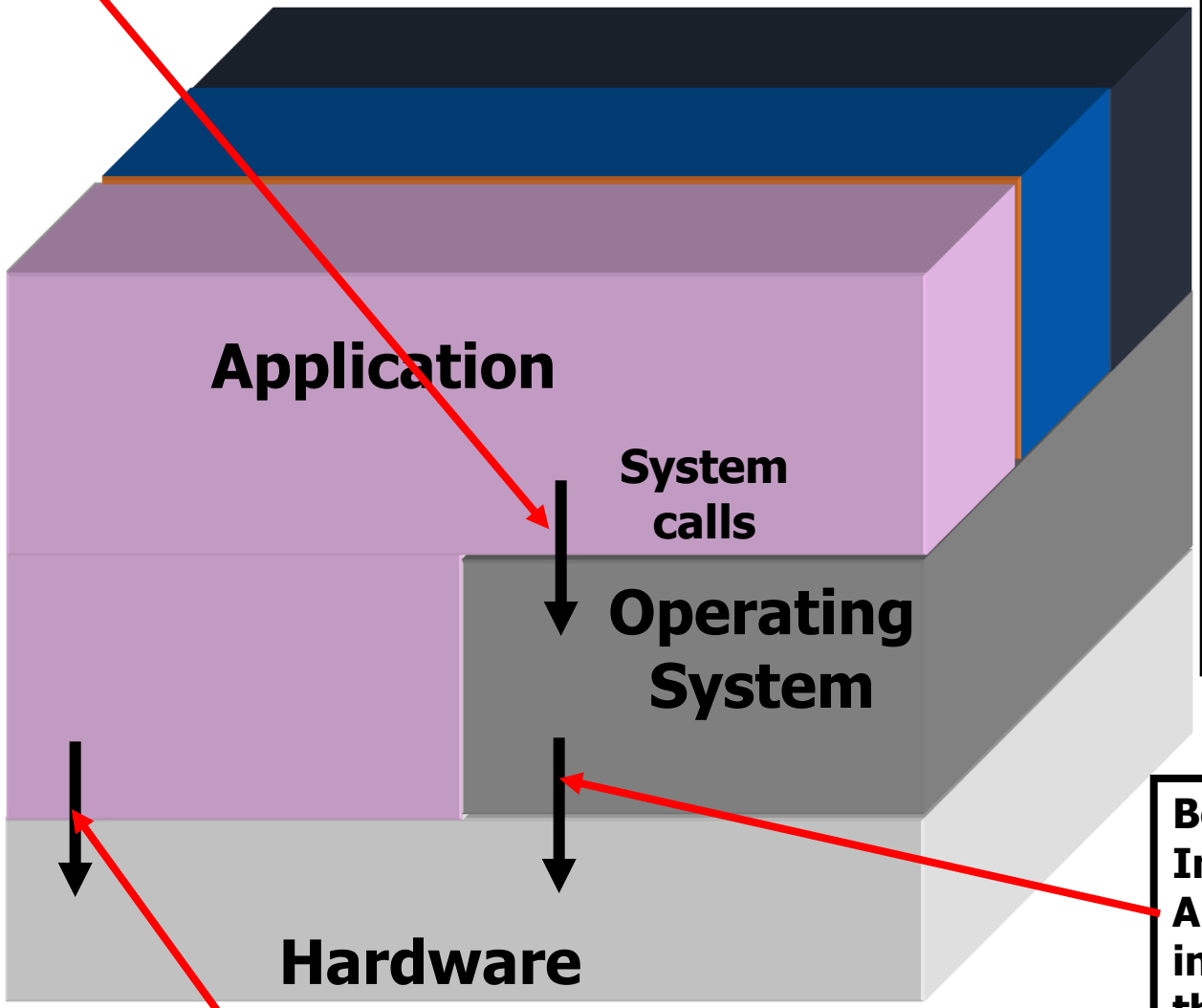
- מצב גרעין (kernel mode). מצב הרשאה גבוהה.
- מצב משתמש (user mode). מצב הרשאה נמוכה.



Both privileged and non privileged Instructions
An example of a privileged instruction: HALT (which causes the CPU to stop working)

Non privileged instructions <u>only</u>. For example: ADD.

The mechanism by which a process request the OS to perform a service for him. For example: open a file



המעבד ממומש באופן כזה:

אם בשלב פענוח קוד ההוראה (= Instruction Decode, ובקיצור: ID) המעבד מזהה שהוא 'מתבקש' לבצע הוראה לא מיוחסת – הוא ישלים הביצוע בין אם הוא נמצא במצב גרעין ובין אם במצב משתמש.

אם, לעומת זאת, בשלב פענוח קוד ההוראה המעבד מזהה שהוא 'מתבקש' לבצע הוראה מיוחסת – <u>הוא ישלים רק אם הוא נמצא במצב גרעין</u>.

אם הוא נמצא במצב משתמש, הוא לא ישלים הביצוע של ההוראה (= 'יסרב לבצע אותה') וייצור פסיקה (interrupt). אשר הטיפול בה יוביל להפסקה מידית של התהליך שהקוד המשויך לו כלל את ההוראה המיוחסת.

{היינו, מצב כזה מתרחש כאשר תהליך משתמש כולל הוראות מיוחסות}

Both privileged and non privileged Instructions
An example of a privileged instruction: HALT (which causes the CPU to stop working)

Non privileged instructions <u>only</u>. For example: ADD.

The mechanism by which a process request the OS to perform a service for him. For example: open a file

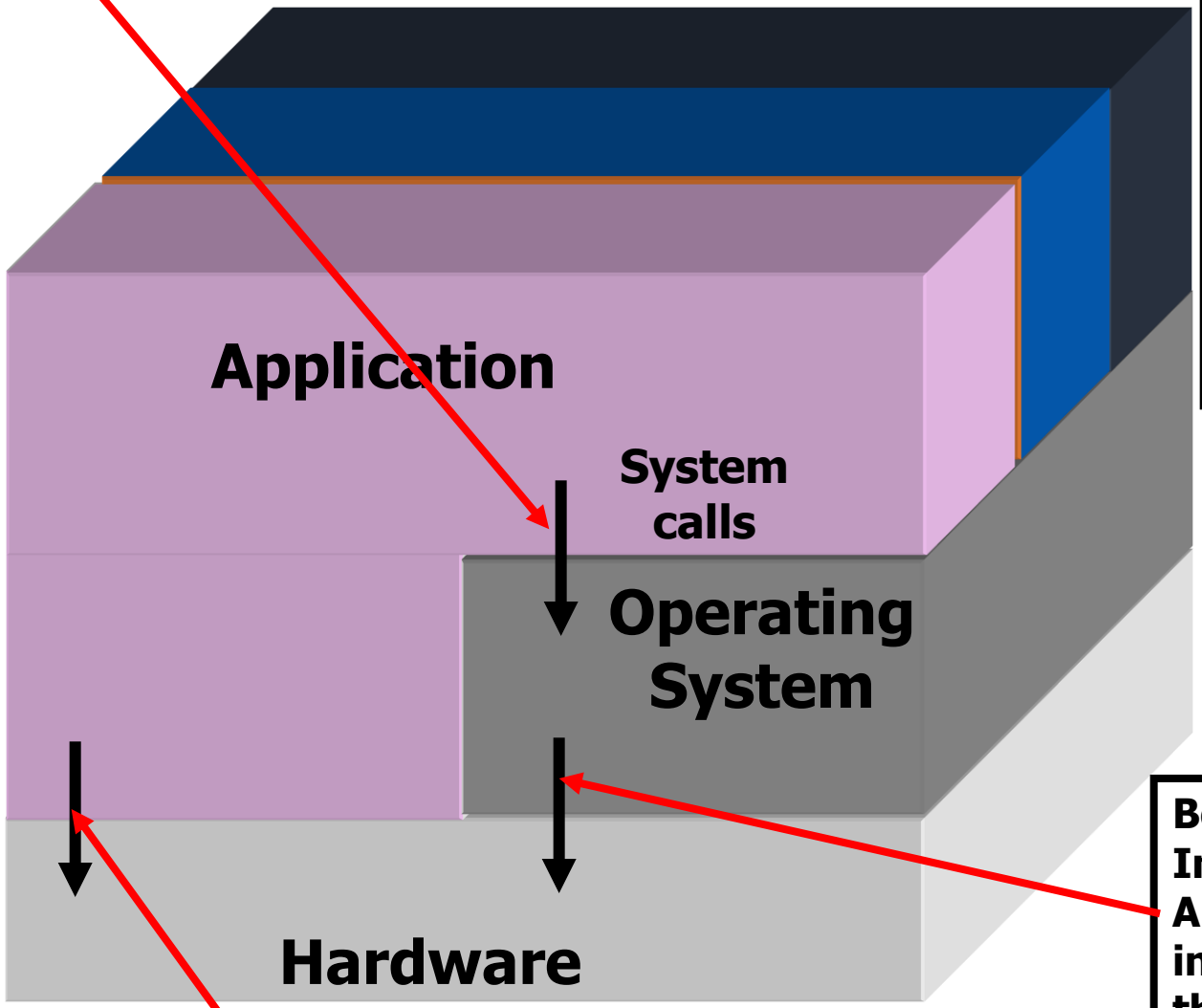
בהמשך לאמור לעיל בשקפים האחרונים, ניתן להגדיר:

הגרעין של מערכת ההפעלה הוא כלל הקוד שכאשר הוא מתבצע המעבד מצוי במצב גרעין.

הקוד של הגרעין כולל כמובן הן הוראות מיוחסות והן לא מיוחסות.

תהליך (או: תכנית) משתמש הוא כל תהליך שכאשר הקוד שמשויך לו מתבצע, המעבד נמצא במצב משתמש.

או בדרך אחרת: כל הקוד שאיננו שייך לקוד של הגרעין.



Both privileged and non privileged Instructions
An example of a privileged instruction: HALT (which causes the CPU to stop working)

Non privileged instructions <u>only</u>. For example: ADD.

Kernel’s modules

Kernel's modules [components]

- Process Management
  - Creation/deletion of processes
  - processes Suspension/resumption
  - Mechanism for communication
  - Deadlock handling

- Main Memory Management
  - Keep track of used and available memory segments
  - Decision which segment will be allocated to a given process
  - Allocate and deallocate memory space

- File Management
  - Creation and deletion of files
  - Creation/deletion of directories
  - Support manipulation file and directories primitives
  - Mapping files into secondary storage

Kernel's modules [components]

- I/O System Management
  - Memory management component including buffering, caching, and spooling
  - General device-driver interface
  - Drivers for specific hardware devices

- Secondary-storage management
  - Free space management
  - Storage allocation
  - Disk scheduling

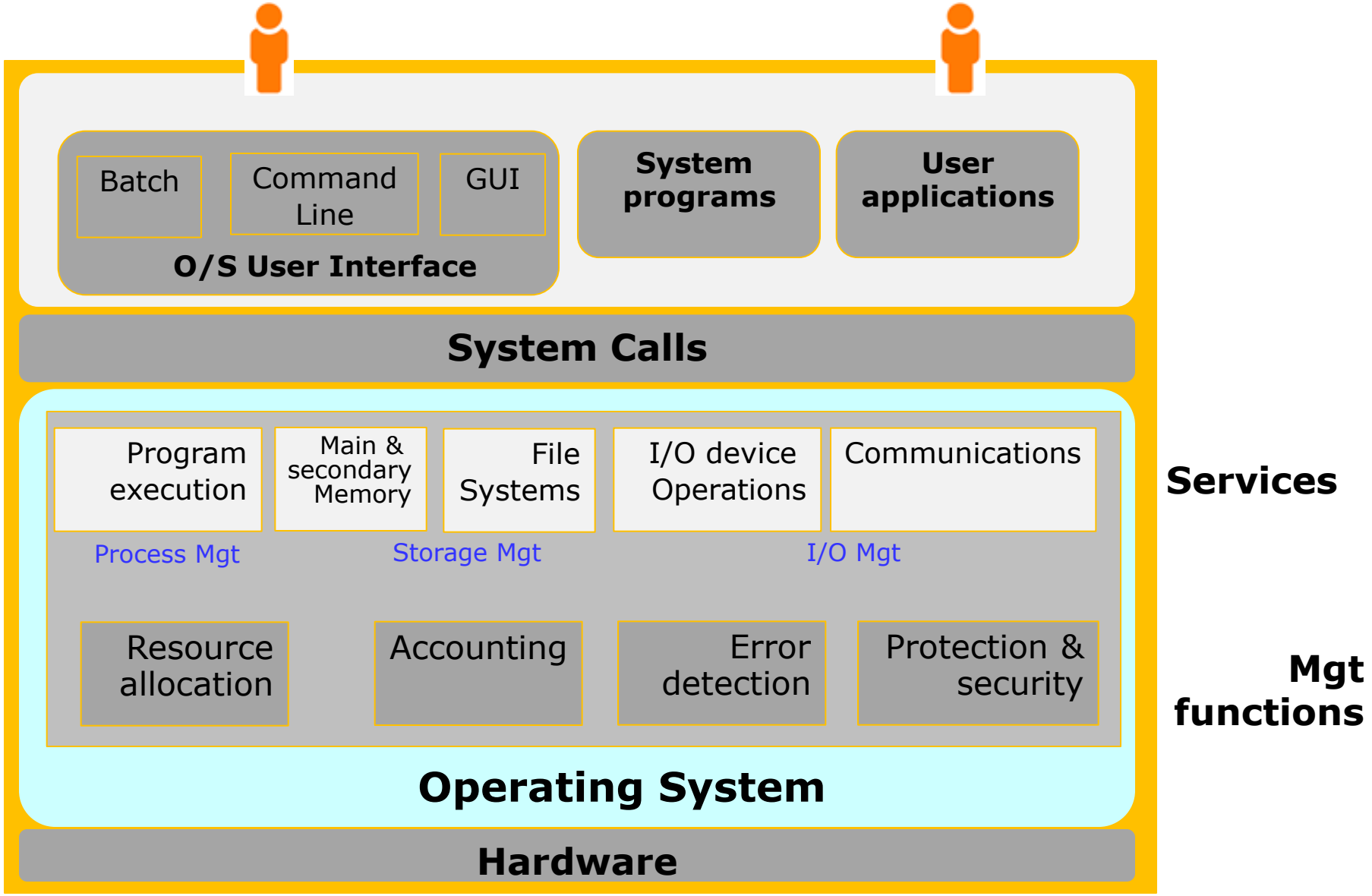
- Networking
  - TCP/IP protocols
  - Memory management component including buffering, caching, and spooling

Kernel's modules [components]

- Protection system
  - Process can act only on those that they are authorized of
  - Protection of process memory address space from other process

- Error and faults handling
- Detection and handling of resources lost or waste
- Detection and handling of over utilization resources (denial of service attacks)

The OS UI


Batch
Command Line
GUI
O/S User Interface
System programs
User applications
System Calls
Program execution
Main & secondary Memory
File Systems
I/O device Operations
Communications
Process Mgt
Storage Mgt
I/O Mgt
Resource allocation
Accounting
Error detection
Protection & security
Operating System
Hardware
Services
Mgt functions

User Interfaces

- One set of programs that are related to the management and use of the OS are user interfaces
  - **User interface** - Almost all operating systems have a user interface (**UI**). Varies between
    - **Batch**
    - **Command-Line** (**CLI**)
    - **Graphics User Interface** (**GUI**)
    - **Voice activated**

User Operating System Interface - CLI

CLI or **command (line) interpreter** allows direct command entry

- Sometimes implemented in kernel, sometimes by systems program (advantages and disadvantages?)
- Sometimes multiple flavors implemented – **shells**
- Primarily fetches a command from user and executes it
- Sometimes commands built-in, sometimes just names of programs
  - If the latter, adding new features does not require shell modification

User Operating System Interface - GUI

- User-friendly **desktop** metaphor interface
  - Usually mouse, keyboard, and monitor
  - **Icons** represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**)
  - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
  - Microsoft Windows is GUI with CLI "command" shell
  - Apple Mac OS X is "Aqua" GUI interface with UNIX kernel underneath and shells available
  - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)